



FP

```
def verifyClass(iface, candidate, tentative=False):
    if not issubclass(candidate, iface):
        return False

    # verify all abstract methods are implemented
    for name in iface.__abstractmethods__:
        m = getattr(candidate, name, None)
        if m is None or not callable(m):
            return False

    if tentative:
        return True

    # ... same pattern for __abstractproperties__ ...
    # ... same pattern for __abstractattributes__ ...

    return True
```

SonarCloud: Rel 0, Maint 2, Sec 0
Lizard: LoC 18, CyC 11, CoC 13

AWQ

```
import inspect

def verifyClass(iface, candidate, tentative=False):
    """
    Verify that the *candidate* might correctly
    provide *iface*.
    """
    if not tentative:
        return issubclass(candidate, iface)

    return all(hasattr(candidate, attr) for attr in dir(iface))
```

SonarCloud: Rel 0, Maint 1, Sec 0
Lizard: LoC 5, CyC 2, CoC 1

GPTQ

```
def verifyClass(iface, candidate, tentative=False):
    """
    Verify that the *candidate* might correctly
    provide *iface*.
    """
    if tentative:
        return all(hasattr(candidate, attr) for attr in
                    iface.__dict__)
    else:
        return all(getattr(candidate, attr) == getattr(
            iface, attr) for attr in iface.__dict__)
```

SonarCloud: Rel 0, Maint 1, Sec 0
Lizard: LoC 5, CyC 2, CoC 2

GGUF

```
def verifyClass(iface, candidate, tentative=False):
    """
    Verify that the *candidate* might correctly
    provide *iface*.
    """
    if tentative:
        return iface in candidate.__dict__
    else:
        return all(hasattr(candidate, attr) for attr in
                    iface.__dict__)
```

SonarCloud: Rel 0, Maint 1, Sec 0
Lizard: LoC 5, CyC 2, CoC 2

BnB

```
import inspect

def verifyClass(iface, candidate, tentative=False):
    """
    Verify that the *candidate* might correctly
    provide *iface*.
    """
    if not tentative:
        return all(hasattr(candidate, attr) and callable(
            getattr(candidate, attr)) for attr in
                    iface.__dict__ if not attr.startswith('__'))
    else:
        return all(hasattr(candidate, attr) for attr in
                    iface.__dict__ if not attr.startswith('_'))
```

SonarCloud: Rel 0, Maint 1, Sec 0
Lizard: LoC 6, CyC 5, CoC 3

AQLM

```
def verifyClass(iface, candidate, tentative=False):
    """
    Verify that the *candidate* might correctly
    provide *iface*.
    """
    if tentative:
        return all(hasattr(candidate, attr) for attr in
                    iface.__dict__)
    else:
        return all(hasattr(candidate, attr) and callable(
            getattr(candidate, attr)) for attr in
                    iface.__dict__)
```

SonarCloud: Rel 0, Maint 1, Sec 0
Lizard: LoC 5, CyC 3, CoC 3

QuIP#

```
def verifyClass(iface, candidate, tentative=False):
    """
    Verify that the *candidate* might correctly
    provide *iface*.
    """
    return isinstance(candidate, iface) or (not tentative
        and not isinstance(candidate, iface))
```

SonarCloud: Rel 0, Maint 1, Sec 0
Lizard: LoC 2, CyC 3, CoC 2